# EEVDF Scheduling Implementation and Extensions in XV6

Operating Systems Course Project

**Team Members**

1. Saumadeep Sardar (Roll No.: CS23B1049)
2. Dhage Pratik Bhishmacharya (Roll No.: CS23B1047)
3. Nisarg Ranade (Roll No.: CS23B1090)

Department of Computer Science
Indian Institute Of Information Technology Design and Manufacturing Kancheepuram

Academic Year: 2025

Source Code

# Contents

# Abstract

This report presents the integration and extension of the Earliest Eligible Virtual Deadline First (EEVDF) scheduler inside the XV6 operating system. Our modifications include proportional weight-based scheduling, virtual runtime tracking, real-time scheduling trace mode, and a user-facing process information reporting system. Full kernel modifications, user programs, and output sections are documented.

# Chapter 1

# Introduction

## 1.1 What is EEVDF?

EEVDF (Earliest Eligible Virtual Deadline First) is an optimal proportional-share CPU scheduling algorithm used in modern operating systems such as Linux's Completely Fair Scheduler (CFS).

Each process is assigned:

- **Weight** — its proportional CPU share.
- **Virtual Runtime** — time consumed adjusted by weight.
- **Virtual Deadline** — determines scheduling order.

## 1.2 Why EEVDF is Better Than Traditional Schedulers

- Provides provably fair CPU distribution.
- Excellent interactivity and latency.
- No starvation.
- Predictable deadline-based selection.
- Weight-based proportional execution sharing.

## 1.3 Core Concepts Relevant in XV6

Before implementing EEVDF, XV6 used a simple round-robin scheduler. By switching to EEVDF, XV6 becomes:

- more fair,
- more predictable,
- more realistic as a teaching OS.

# Chapter 2

# Design Overview

## 2.1 EEVDF Scheduling Formulae

Egalitarian Earliest Virtual Deadline First (EEVDF) is a proportional-fair scheduler that assigns each process a virtual runtime and a virtual deadline. Tasks with smaller virtual deadlines are scheduled first.

### 2.1.1 Virtual Runtime Update

The virtual runtime increases proportionally to how long a process executes and inversely proportional to its weight:

$$\Delta v = \frac{\Delta t \times 2^{\text{FIXED\_SHIFT}}}{\text{weight}}$$

$$v_{\text{runtime,new}} = v_{\text{runtime,old}} + \Delta v$$

### 2.1.2 Global Virtual Time

The system keeps a monotonically increasing virtual time:

$$V_{\text{global}} = \max(V_{\text{global}},\ v_{\text{runtime}})$$

### 2.1.3 Eligibility Rule

A process becomes eligible when:

$$v_{\text{runtime}} \leq V_{\text{global}}$$

### 2.1.4 Virtual Deadline Calculation

The scheduler computes the virtual deadline as:

$$v_{\text{deadline}} = v_{\text{runtime}} + \frac{S_{\text{FIXED}}}{\text{weight}}$$

This corresponds to the EEVDF formula:

$$D = v + \frac{q}{w}$$

### 2.1.5 Scheduler Selection Rule

The scheduler selects:

$$p = \arg\min\_p \left(v_{\text{deadline}}\right)$$

over all eligible runnable processes.

## 2.2 Data Structure Changes (kernel/proc.h)

### 2.2.1 Fields Added to `struct proc`

Listing 2.1 shows all the fields added to support EEVDF scheduling, trace logging and runtime statistics.

Listing 2.1: Additional fields in struct proc (kernel/proc.h)

```
uint64 vruntime;         // EEVDF virtual runtime
uint64 vdeadline;        // EEVDF virtual deadline
int    weight;           // Scheduling weight

uint64 runtime;          // Total CPU time used (ticks)
uint64 sched_count;      // Number of times scheduled
uint64 sleep_time;       // Time spent sleeping
uint64 wait_time;        // Time spent runnable but not scheduled
uint64 last_scheduled;   // Last time it ran (for wait_time
   calculation)
```

### 2.2.2 Fields Added to `struct pinfo`

Listing 2.2 shows the structure returned to user space via `getpinfo()`.

Listing 2.2: struct pinfo exposed to user space

```
struct pinfo {
  int pid;
  int state;
  int weight;
  uint64 runtime;
  uint64 sched_count;
  uint64 vdeadline;
};
int getpinfo(struct pinfo *info);
```

Additionally, the global flag for scheduling trace output is:

```
extern int sched_trace;
```

# Chapter 3

# Kernel Modifications

## 3.1 Complete EEVDF Scheduler Implementation (kernel/proc.c)

Listing 3.1: EEVDF Scheduler Function in kernel/proc.c

```
void
scheduler(void)
{
  struct cpu *c = mycpu();
  c->proc = 0;

  for(;;){
    intr_on();  // enable interrupts to prevent deadlock

    // ---- Print any deferred event for this CPU (outside locks)
    int cid = cpuid();
    if (sched_events[cid].valid) {
      struct sched_event e = sched_events[cid];
      sched_events[cid].valid = 0;
      printf("[EEVDF] CPU=%d pid=%d weight=%d vdeadline=%llu\n",
             e.cpu, e.pid, e.weight, (unsigned long long)e.
                vdeadline);
    }

    struct proc *p = 0;
    uint64 best_deadline = (uint64)-1;

    // Step 1: find the runnable process with the smallest
       eligible vdeadline
```

```
for(struct proc *q = proc; q < &proc[NPROC]; q++) {
  acquire(&q->lock);
  if(q->state == RUNNABLE) {
    if(q->eligible_time <= global_vtime && q->vdeadline <
       best_deadline) {
      if(p) release(&p->lock); // release previous candidate's
          lock
      p = q;                   // p now holds q->lock
      best_deadline = q->vdeadline;
    } else {
      release(&q->lock);
    }
  } else {
    release(&q->lock);
  }
}

if(p == 0) {
  // No process is runnable     wait for interrupt
  intr_on();
  asm volatile("wfi");
  continue;
}

// At this point `p` is selected and its lock is held.

// Step 2: run the chosen process (we hold p->lock)
p->state = RUNNING;
c->proc = p;

// record ticks before running
int before_ticks = p->run_ticks;

// context switch to process (p->lock stays held across swtch
   in xv6 convention)
swtch(&c->context, &p->context);

// Returned here after the process yielded or was preempted.
c->proc = 0;

// Still holding p->lock here (as in xv6 convention).
```

```
// Step 3: process has yielded or been preempted, update EEVDF
    & stats

// compute how many real ticks it ran
int ran_ticks = p->run_ticks - before_ticks;
if (ran_ticks < 0) ran_ticks = 0;

// Convert to fixed-point delta time and update vruntime
uint64 delta_v = ((uint64)ran_ticks << FIXED_SHIFT) / (p->
   weight > 0 ? p->weight : 1);
p->vruntime += delta_v;

// Update global virtual time if needed (simple policy)
if(p->vruntime > global_vtime)
  global_vtime = p->vruntime;

// Update eligibility and new deadline
p->eligible_time = p->vruntime;
p->vdeadline = p->eligible_time + (S_FIXED / (uint64)(p->
   weight > 0 ? p->weight : 1));

// Update runtime/sched counters (use ticks and last_scheduled
    if you set them)
// If last_scheduled is used, ensure it was set when the
   process actually started
// Here we update simple runtime and sched_count (you can
   refine as needed)
p->sched_count++;
// You may track runtime differently; if you set
   last_scheduled before running, use that.
// For simplicity, increment runtime by ran_ticks:
p->runtime += ran_ticks;

// Record a deferred trace event (still while holding p->lock
   is fine because we only write CPU-local slot)
if (sched_trace) {
  int cid2 = cpuid();
  sched_events[cid2].cpu = cid2;
  sched_events[cid2].pid = p->pid;
  sched_events[cid2].weight = p->weight;
  sched_events[cid2].vdeadline = p->vdeadline;
```

```
      // publish last
      sched_events[cid2].valid = 1;
    }

    // Step 4: release p->lock and continue scheduling loop
    release(&p->lock);
  }
}
```

## 3.2 Code Snippet: Added Functions in `kernel/proc.c`

Listing 3.2: EEVDF helper functions added in proc.c

```
void
setweight(int w) {
  struct proc *p = myproc();
  acquire(&p->lock);
  if(w > 0)
    p->weight = w;
  release(&p->lock);
}

int
getpinfo(struct pinfo *info)
{
  struct proc *p = myproc();

  if (!info)
    return -1;

  acquire(&p->lock);
  info->pid = p->pid;
  info->state = p->state;
  info->weight = p->weight;
  info->runtime = p->runtime;
  info->sched_count = p->sched_count;
  info->vdeadline = p->vdeadline;
  release(&p->lock);

  return 0;
}
```

## 3.3 System Call: setweight() (kernel/sysproc.c)

Listing 3.3: sys_setweight implementation

```
uint64
sys_setweight(void)
{
  int w;
  argint(0, &w);
  setweight(w);
  return 0;
}
```

## 3.4 System Call: setschedtrace() (kernel/sysproc.c)

Listing 3.4: sys_setschedtrace implementation

```
uint64 sys_setschedtrace(void)
{
  int enable;
  argint(0, &enable);
  sched_trace = enable;
  return 0;
}
```

## 3.5 System Call: pinfo() (kernel/sysproc.c)

Listing 3.5: sys_pinfo implementation

```
uint64
sys_getpinfo(void)
{
  uint64 addr;
  struct pinfo info;
  argaddr(0, &addr);
  getpinfo(&info);
  return copyout(myproc()->pagetable, addr, (char*)&info, sizeof(
     info));
}
```

# Chapter 4

# User-Level Functions and Test Programs

## 4.1 User API Definitions (user/user.h)

Listing 4.1: New user library calls

```
int setweight(int);
struct pinfo { int pid, state, weight; uint64 runtime, sched_count
   , vdeadline; };
int getpinfo(struct pinfo*);
void setschedtrace(int);
```

## 4.2 Example 1 — eevdf_test1.c: CPU Load Test (no yield)

Listing 4.2: user/eevdf_test.c

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

void heavy_compute(int n) {
  volatile int x = 0;
  for (int i = 0; i < n * 1000000; i++)
    x += i ^ (x << 1);
}

int
```

```
main(void)
{
  printf("==== EEVDF CPU Load Test (no yield) ====\n");
  setschedtrace(1);

  int weights[3] = {1, 3, 5};
  int nchild = 3;

  for (int i = 0; i < nchild; i++) {
    int pid = fork();
    if (pid == 0) {
      setweight(weights[i]);
      int start = uptime();
      heavy_compute(50 - i * 10);  // different workloads
      int end = uptime();
      printf("Child(pid=%d, weight=%d) finished in %d ms\n",
             getpid(), weights[i], (end - start) * 10);
      exit(0);
    }
  }

  for (int i = 0; i < nchild; i++)
    wait(0);

  setschedtrace(0);

  struct pinfo info;
  if (getpinfo(&info) == 0)
    printf("Parent(pid=%d): weight=%d runtime=%ld vdeadline=%ld
       sched_count=%ld\n",
           info.pid, info.weight, info.runtime, info.vdeadline,
              info.sched_count);

  printf("==== Test complete ====\n");
  exit(0);
}
```

Output:

```
$ eevdf_test1
==== EEVDF CPU Load Test (no yield) ====
[EEVDF] CPU=0 pid=3 weight=1 vdeadline=4096
[EEVDF] CPU=1 pid=6 weight=5 vdeadline=1638
[EEVDF] CPU=2 pid=4 weight=1 vdeadline=8192
[EEVDF] CPU=0 pid=5 weight=3 vdeadline=2730
[EEVDF] CPU=1 pid=4 weight=1 vdeadline=12288
[EEVDF] CPU=2 pid=6 weight=5 vdeadline=2457
[EEVDF] CPU=0 pid=5 weight=3 vdeadline=4095
Child(pid=6, weight=5) finished in 20 ms
[EEVDF] CPU=1 pid=6 weight=5 vdeadline=2457
[EEVDF] CPU=1 pid=3 weight=1 vdeadline=4096
Child(pid=5, weight=3) finished in 20 ms
[EEVDF] CPU=0 pid=5 weight=3 vdeadline=4095
[EEVDF] CPU=1 pid=3 weight=1 vdeadline=4096
[EEVDF] CPU=2 pid=4 weight=1 vdeadline=16384
[EEVDF] CPU=0 pid=4 weight=1 vdeadline=20480
Child(pid=4, weight=1) finished in 30 ms
[EEVDF] CPU=0 pid=4 weight=1 vdeadline=20480
Parent(pid=3): weight=1 runtime=0 vdeadline=20480 sched_count=8
==== Test complete ====
```

## 4.3 Example 2 — eevdf_test2.c: Mixed CPU + Sleep Test

:

Listing 4.3: user/weight_demo.c

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

void cpu_burst(int n) {
  volatile int x = 1;
  for (int i = 0; i < n * 1000000; i++)
    x = x * 3 + 1;
}

int
main(void)
{
  printf("==== EEVDF Mixed CPU + Sleep Test ====\n");
  setschedtrace(1);

  int weights[3] = {2, 4, 8};
  int nchild = 3;
```

```
  for (int i = 0; i < nchild; i++) {
    int pid = fork();
    if (pid == 0) {
      setweight(weights[i]);

      int start = uptime();
      for (int k = 0; k < 10; k++) {
        cpu_burst(10);      // Short CPU burst
        sleep(1);           // Voluntary yield replacement
      }
      int end = uptime();

      printf("[Child %d] weight=%d finished in %d ms\n",
             getpid(), weights[i], (end - start) * 10);
      exit(0);
    }
  }

  for (int i = 0; i < nchild; i++)
    wait(0);

  setschedtrace(0);

  struct pinfo info;
  if (getpinfo(&info) == 0)
    printf("Parent(pid=%d): weight=%d runtime=%ld vdeadline=%ld
       sched_count=%ld\n",
           info.pid, info.weight, info.runtime, info.vdeadline,
              info.sched_count);

  printf("==== Test complete ====\n");
  exit(0);
}
```

Output:

```
[EEVDF] CPU=1 pid=8 weight=2 vdeadline=12288
[EEVDF] CPU=0 pid=9 weight=4 vdeadline=5120
[EEVDF] CPU=0 pid=9 weight=4 vdeadline=5120
[EEVDF] CPU=1 pid=8 weight=2 vdeadline=12288
[EEVDF] CPU=2 pid=10 weight=8 vdeadline=1024
[EEVDF] CPU=0 pid=10 weight=8 vdeadline=1024
[EEVDF] CPU=1 pid=9 weight=4 vdeadline=6144
[EEVDF] CPU=2 pid=8 weight=2 vdeadline=14336
[EEVDF] CPU=0 pid=8 weight=2 vdeadline=16384
[EEVDF] CPU=1 pid=9 weight=4 vdeadline=6144
[EEVDF] CPU=0 pid=8 weight=2 vdeadline=16384
[EEVDF] CPU=2 pid=10 weight=8 vdeadline=1024
[EEVDF] CPU=0 pid=10 weight=8 vdeadline=1024
[EEVDF] CPU=1 pid=8 weight=2 vdeadline=18432
[EEVDF] CPU=2 pid=9 weight=4 vdeadline=7168
[EEVDF] CPU=0 pid=8 weight=2 vdeadline=20480
[Child 10] weight=8 finished in 110 ms
[EEVDF] CPU=2 pid=10 weight=8 vdeadline=1024
[EEVDF] CPU=2 pid=7 weight=1 vdeadline=4096
[EEVDF] CPU=1 pid=9 weight=4 vdeadline=7168
[EEVDF] CPU=0 pid=8 weight=2 vdeadline=20480
[EEVDF] CPU=0 pid=9 weight=4 vdeadline=7168
[EEVDF] CPU=1 pid=8 weight=2 vdeadline=22528
[EEVDF] CPU=0 pid=8 weight=2 vdeadline=24576
[EEVDF] CPU=2 pid=8 weight=2 vdeadline=24576
[EEVDF] CPU=0 pid=9 weight=4 vdeadline=7168
[EEVDF] CPU=0 pid=9 weight=4 vdeadline=7168
[EEVDF] CPU=2 pid=8 weight=2 vdeadline=26624
[EEVDF] CPU=1 pid=8 weight=2 vdeadline=26624
[EEVDF] CPU=0 pid=9 weight=4 vdeadline=7168
[Child 9] weight=4 finished in 160 ms
[EEVDF] CPU=0 pid=9 weight=4 vdeadline=7168
[EEVDF] CPU=2 pid=7 weight=1 vdeadline=4096
[EEVDF] CPU=1 pid=8 weight=2 vdeadline=28672
[EEVDF] CPU=1 pid=8 weight=2 vdeadline=28672
[EEVDF] CPU=0 pid=8 weight=2 vdeadline=28672
[Child 8] weight=2 finished in 180 ms
[EEVDF] CPU=0 pid=8 weight=2 vdeadline=28672
Parent(pid=7): weight=1 runtime=0 vdeadline=30720 sched_count=17
==== Test complete ====
```

## 4.4 Example 3 — eevdf_test3.c: Sleep/Wakeup Fairness Test

Listing 4.4: user/pinfo_demo.c

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

void compute_small(int n) {
  volatile int acc = 0;
```

```
  for (int i = 0; i < n * 500000; i++)
    acc ^= (i * acc) + 1;
}

int
main(void)
{
  printf("==== EEVDF Sleep/Wakeup Fairness Test ====\n");
  setschedtrace(1);

  int weights[3] = {1, 2, 4};
  int nchild = 3;

  for (int i = 0; i < nchild; i++) {
    int pid = fork();
    if (pid == 0) {
      setweight(weights[i]);

      int start = uptime();
      for (int iter = 0; iter < 5; iter++) {
        printf("[PID=%d, W=%d] Iter=%d: sleeping...\n",
               getpid(), weights[i], iter);
        sleep(20);              // simulate I/O-bound behavior
        compute_small(8 + i);  // small compute burst
      }
      int end = uptime();

      printf("Child(pid=%d, weight=%d) completed sleep/compute
         pattern in %d ms\n",
             getpid(), weights[i], (end - start) * 10);

      exit(0);
    }
  }

  for (int i = 0; i < nchild; i++)
    wait(0);

  setschedtrace(0);

  struct pinfo info;
```

```
  if (getpinfo(&info) == 0)
    printf("Parent(pid=%d): weight=%d runtime=%ld vdeadline=%ld
       sched_count=%ld\n",
           info.pid, info.weight, info.runtime, info.vdeadline,
              info.sched_count);

  printf("==== Test complete ====\n");
  exit(0);
}
```

Output:

```
$ eevdf_test3
==== EEVDF Sleep/Wakeup Fairness Test ====
[EEVDF] CPU=0 pid=11 weight=1 vdeadline=4096
[[PPIIDD==[1134,,  WW==24]]  IItteerr==00::  sslleeeeppiinngg......

[EEVDF] CPU=0 pid=13 weight=2 vdeadline=2048
[EEVDF] CPU=1 pid=14 weight=4 vdeadline=1024
PID=12, W=1] Iter=0: sleeping...
[EEVDF] CPU=2 pid=12 weight=1 vdeadline=4096
[EEVDF] CPU=0 pid=14 weight=4 vdeadline=1024
[EEVDF] CPU=2 pid=13 weight=2 vdeadline=2048
[EEVDF] CPU=1 pid=12 weight=1 vdeadline=4096
[EEVDF] CPU=0 pid=14 weight=4 vdeadline=1024
[EEVDF] CPU=1 pid=13 weight=2 vdeadline=2048
[EEVDF] CPU=2 pid=12 weight=1 vdeadline=4096
[EEVDF] CPU=0 pid=14 weight=4 vdeadline=1024
[EEVDF] CPU=1 pid=13 weight=2 vdeadline=2048
[EEVDF] CPU=2 pid=12 weight=1 vdeadline=4096
[EEVDF] CPU=0 pid=14 weight=4 vdeadline=1024
[EEVDF] CPU=0 pid=13 weight=2 vdeadline=2048
[EEVDF] CPU=2 pid=12 weight=1 vdeadline=4096
[EEVDF] CPU=0 pid=14 weight=4 vdeadline=1024
[EEVDF] CPU=0 pid=13 weight=2 vdeadline=2048
[EEVDF] CPU=2 pid=12 weight=1 vdeadline=4096
[EEVDF] CPU=0 pid=14 weight=4 vdeadline=1024
[EEVDF] CPU=2 pid=13 weight=2 vdeadline=2048
[EEVDF] CPU=1 pid=12 weight=1 vdeadline=4096
[EEVDF] CPU=0 pid=14 weight=4 vdeadline=1024
[EEVDF] CPU=2 pid=13 weight=2 vdeadline=2048
[EEVDF] CPU=1 pid=12 weight=1 vdeadline=4096
[EEVDF] CPU=0 pid=14 weight=4 vdeadline=1024
[EEVDF] CPU=1 pid=13 weight=2 vdeadline=2048
[EEVDF] CPU=2 pid=12 weight=1 vdeadline=4096
[EEVDF] CPU=0 pid=14 weight=4 vdeadline=1024
[EEVDF] CPU=1 pid=13 weight=2 vdeadline=2048
[EEVDF] CPU=2 pid=12 weight=1 vdeadline=4096
[EEVDF] CPU=0 pid=14 weight=4 vdeadline=1024
[EEVDF] CPU=2 pid=13 weight=2 vdeadline=2048
[EEVDF] CPU=1 pid=12 weight=1 vdeadline=4096
[EEVDF] CPU=0 pid=14 weight=4 vdeadline=1024
[EEVDF] CPU=1 pid=13 weight=2 vdeadline=2048
[EEVDF] CPU=0 pid=12 weight=1 vdeadline=4096
[EEVDF] CPU=0 pid=14 weight=4 vdeadline=1024
[EEVDF] CPU=1 pid=13 weight=2 vdeadline=2048
```

# Chapter 5

# Conclusion

We successfully implemented EEVDF scheduling inside XV6 with full system call support, tracing, proportional fairness, user-space observability, and validation through multiple test programs. This redesign makes XV6 behave like a miniature modern Unix kernel.

# Chapter 6

# Future Work

- Multi-core aware EEVDF scheduling.
- Deadline-controlled I/O scheduling.
- GUI visualizer for vruntime evolution.

# Chapter 7

# Bibliography


- MIT 6.S081 XV6 Textbook and Source
- Linux 5.x CFS Documentation
- Waldspurger, C. “Lottery and Stride Scheduling”
- Silberschatz, Galvin — Operating System Concepts